Relatório — Trabalho 4: Driver PS/2

Lucas Damo · Antônio dos Santos

Projeto de Sistemas Integrados (98G07-04) · Turma 310 · 06/11/2026

1. Visão geral

Projetamos o bloco `ps2_driver` em SystemVerilog. Ele lê um teclado pelo protocolo PS/2 e entrega o caractere em ASCII em `data_p`, com `data_p_en` em nível alto durante um ciclo de `clock_in`. O RTL foi verificado no QuestaSim 2025.3 com um testbench autoverificável e sintetizado no Cadence Genus 21.1 com a biblioteca ASAP7 (7 nm) usada no laboratório 2.

2. Arquitetura

Toda a lógica é sensível à borda de subida de `clock_in` (100 MHz) e usa reset assíncrono ativo em nível alto (item 6 da especificação; a Tabela 1 diz “síncrono”, mas seguimos o item 6). O caminho de dados é:

clock_ps2, data_ps2 → sincronizador 2-FF → filtro de glitch → borda de descida → FSM → filtro F0 → decodificador → data_p, data_p_en

- **Sincronizador:** `clock_ps2` e `data_ps2` vêm do teclado, sem relação com `clock_in`, e passam por 2 flip-flops. O valor de reset é 1, o nível ocioso do PS/2.
- **Filtro de glitch:** o `clock_ps2` filtrado só muda após `FILTER` = 8 amostras iguais seguidas (80 ns), bem abaixo do semiperíodo mínimo de 500 ns a 1 MHz. Um glitch não gera um bit falso.
- **Detector de borda:** gera o pulso `fall` de 1 ciclo na borda de descida do clock filtrado; a FSM só avança nesse pulso, amostrando `data_ps2` como pede a Figura 2.
- **Filtro de break code:** o byte `F0` indica que a próxima tecla foi solta; esse byte é descartado, então cada tecla gera um único caractere.
- **Decodificador:** um `case` combinacional mapeia 52 scancodes da Figura 3: A–Z (maiúsculas), 0–9, espaço, os 11 símbolos sem shift e Enter (0x0D), Backspace (0x08), Tab (0x09) e Esc (0x1B). Teclas sem ASCII (F1–F12, Shift, Ctrl, Alt, Caps) não geram saída.
- **Saída:** `data_p` é registrado e mantém o último caractere; `data_p_en` fica em 1 só no ciclo seguinte a uma decodificação válida.

Estado	Ação em cada borda de descida de clock_ps2
IDLE	se data = 0 (start bit) → DATA
DATA	desloca o bit para o registrador, LSB primeiro; após 8 bits → PARITY
PARITY	par_ok = XOR dos 8 bits e de P (paridade ímpar: o XOR dos 9 bits é 1) → STOP
STOP	se stop = 1 e par_ok, entrega o byte; sempre → IDLE

Simplificações: não tratamos Shift/Caps Lock (letras sempre maiúsculas, símbolos sem shift) nem o prefixo `E0` (é ignorado e o byte seguinte é decodificado normalmente). Não há timeout de quadro: um quadro cortado no meio desalinha a recepção até o próximo reset.

3. Verificação

O testbench (`tb.sv`) gera os quadros PS/2 com a tarefa `send`: o dado muda com `clock_ps2` em alto e o clock desce meio período depois. Um scoreboard guarda numa fila o ASCII esperado e compara cada `data_p` quando `data_p_en` sobe. Uma assertion concorrente (`data_p_en |=> !data_p_en`) garante o pulso de um ciclo. Testes aplicados:

Teste	Item	Resultado esperado
As 52 teclas mapeadas, a 1 MHz e a 100 kHz	7, 11	data_p = ASCII da tecla
Tecla pressionada e solta: 1C, F0 1C	—	um único 'A'
Quadro com paridade errada	9	nenhuma saída
Quadro com stop bit errado	10	nenhuma saída
Teclas sem ASCII: F1, Shift, Caps	11	nenhuma saída
Glitch de 30 ns em clock_ps2 com data em 0	—	não é tomado como start bit
Pulso de reset de 2 ns entre bordas de clock_in, no meio de um quadro	6	quadro descartado; o seguinte é decodificado
data_p_en em todos os testes	8	nunca em 1 por dois ciclos seguidos

Resultado: `PASS`, 0 erros em 7,03 ms de tempo simulado (ver `out.log`).

Para validar o próprio testbench, inserimos 12 defeitos no RTL, um de cada vez. O testbench detectou todos:

Defeito inserido no RTL	Erros	Defeito inserido no RTL	Erros
reset síncrono	1	break code ignorado	1
paridade par	2	sem checagem de paridade	1
bits recebidos MSB primeiro	35	sem checagem do stop bit	1
amostra na borda de subida	12	'A' mapeado para 'a'	3
sem filtro de glitch	2	'-' mapeado para '_'	2
data_p_en preso em 1	1.403.879	Enter mapeado para 0x0A	2

4. Síntese lógica

O script `synth/scripts/genus_t4.tcl` segue a estrutura dos scripts do laboratório 2: mesmas bibliotecas ASAP7 LVT, corner TT (0,7 V, 25 °C), LEFs escalados 4x e arquivo QRC, com inserção de clock gating ativada. As constraints estão em `synth/constraints/ps2_driver.sdc` (unidades da biblioteca: ps e fF):

Constraint	Valor	Motivo
create_clock clock_in	10 000 ps	100 MHz (item 4)
set_clock_uncertainty / transition	100 ps / 20 ps	margem para jitter e skew
set_input_delay (reset_in, clock_ps2, data_ps2)	3 000 ps	30% do período, como no lab 2
set_output_delay (data_p, data_p_en)	3 000 ps	30% do período
set_false_path -from clock_ps2, data_ps2	—	assíncronos, entram no sincronizador 2-FF
set_driving_cell / set_load	BUFx2 / 2 fF	entradas e saídas realistas

O `check_design` não encontrou referências não resolvidas e o `report timing -lint` não apontou nenhum problema (nenhuma entrada ou saída sem constraint).

5. Resultados

Timing (`ps2_driver.timing.rep` e `ps2_driver.timing_reg2reg.rep`): todos os caminhos cumprem a restrição de 100 MHz com folga.

Caminho	Atraso	Slack
Pior caminho geral: data_p_en_reg → porta data_p_en (com 3 000 ps de output delay)	52 ps	+6 848 ps
Pior caminho interno: sr_reg[5] → decodificador → enable do clock gating	407 ps	+9 463 ps

Área (`ps2_driver.area.rep` e `ps2_driver.gates.rep`):

Tipo	Células	Área (μm²)
Flip-flops (DFFASRHQNx1, reset/set assíncrono)	37	224,415
Clock gating (ICGx1)	4	16,796
Inversores	51	35,692
Lógica combinacional	137	180,092
Total de células	229	456,996
Interconexão (estimada)	—	181,740
Área total		**638,735**

O RTL tem 38 flip-flops, mas o Genus removeu `data_p[7]`: todo ASCII mapeado é menor que 0x80, então esse bit é sempre 0. O clock gating cobre 30 dos 37 flip-flops (81%).

Potência (`ps2_driver.power.rep`, estimativa vectorless do Genus, a 100 MHz):

Categoria	Leakage	Interna	Chaveamento	Total	%
Registradores	0,110	2,856	0,233	3,200	61,0
Lógica	0,120	0,263	0,450	0,832	15,9
Clock	0,012	0,303	0,895	1,209	23,1
Total (μW)	0,242	3,422	1,578	**5,241**	100

A potência total é de 5,24 μW. Os registradores respondem por 61% do total, quase toda em potência interna das células.

6. Arquivos entregues

- `ps2_driver.sv`: RTL do bloco.
- `tb.sv`, `ps2_driver_if.sv`, `sim.do`, `out.log`: testbench, interface, script e log da simulação (`vsim -c -do "do sim.do; quit -f"`).
- `synth/scripts/genus_t4.tcl` e `synth/constraints/ps2_driver.sdc`: script de síntese e constraints. Executar em `synth/run` com `genus -batch -files ../scripts/genus_t4.tcl`, com as pastas `lib`, `lef`, `techlef` e `qrc` do laboratório 2 dentro de `synth/`.
- `synth/reports/`: relatórios de timing, área, portas, potência, clocks, lint e clock gating.
- `synth/outputs/`: netlist sintetizada e SDC gerado pelo Genus.